Universidad Nacional de Colombia - sede Bogotá
School of Engineering
Software Engineering II

Workshop 2

Cristian Camilo Ochica Maldonado
Joan Sebastian Roberto Puerto
Juan David Martinez Herrera
Santiago Cardenas Rodriguez
Alejandro Rodriguez Arciniegas

1. System Domain

System Context

The system is an interactive programming learning platform designed to teach fundamental coding concepts (such as Variables, Conditionals, Loops, Arrays, and Functions) through a gamified, hands-on approach. The scope of the system includes a web-based code editor, an automated evaluation engine (virtual judge), an interactive learning roadmap, and gamification mechanics (XP, daily streaks, and a weekly leaderboard) to increase student retention. The platform does not cover general-purpose Learning Management System (LMS) functionalities like grading essays or managing course enrollment, but focuses exclusively on assessing and verifying source code automatically using secure containers. The main goal is to provide a comprehensive, micro-learning programming environment that reduces frustration and relies on self-hosted architecture for autonomous evaluation.

Actor Characterization

- **Student:** The primary end-user of the platform. Their main goal is to navigate the learning roadmap, write and submit code in multiple programming languages (primarily C++), use the browser-based editor, earn XP points, and track their progress via the leaderboard and analytics dashboard

- **Administrator / Teacher:** The manager of the platform’s content and technical constraints. Their main goal is to create, edit, and organize learning modules and problem sets, configure automated test cases, manage user roles, and define execution limits (memory and time) for the virtual judge.
- **Virtual Judge / Execution Engine (System Actor):** An internal automated module responsible for receiving the student's code, compiling it, executing it against predefined hidden test cases in secure containers, and returning a verdict (e.g., Accepted, Compilation Error, Time Limit Exceeded) along with resource consumption metrics.

Business Rules

1. **Authentication Policy:** All student traditional passwords must be encrypted in the database. Third-party authentication (Google) must exclusively use the OAuth 2.0 protocol.
2. **Execution Limits Enforcement:** The automated evaluation engine must strictly enforce the maximum execution time and memory limits configured by the administrator for each specific problem. If a student's code exceeds these limits, the execution must be forcefully interrupted and return the corresponding error (Time limit exceeded - TLE or Memory limit exceeded - MLE)
3. **Module Unlocking Condition:** A student cannot access a locked learning module unless they meet the prerequisite success rate (e.g., 80% effectiveness) on the preceding topics, as configured by the administrator.
4. **Gamification Constraints:** Students are allowed unlimited submission attempts per exercise without losing "lives" or energy. However, the weekly leaderboard and its associated counters must automatically reset at the end of every week via background cron jobs to maintain a competitive environment, while keeping the overall XP intact.
5. **Test Case Privacy:** Under no circumstances should the system reveal the "hidden" test cases (input/output parameters) of an exercise to the student during the evaluation process.

Part 2: CRC Cards

1. Class: Student

Responsibilities	Collaborators
● Authenticates via OAuth 2.0 or traditional email/password. ● Navigates the interactive learning roadmap and views theoretical material. ● Writes, tests (draft mode), and submits code for evaluation. ● Tracks academic progress, XP, and daily streaks.	● GamificationProfile ● Submission ● Module

2. Class: Administrator

Responsibilities	Collaborators
● Manages user roles and platform permissions. ● Creates, edits, and organizes learning modules and exercises. ● Configures execution limits (time and memory) and supported languages. ● Manages public and hidden test cases for evaluation.	● Module ● Exercise ● TestCase

3. Class: Exercise (Problem)

Responsibilities	Collaborators
● Stores problem description, difficulty, and thematic tags. ● Maintains specific technical limits (execution time, maximum memory). ● Groups public and hidden test cases.	● TestCase ● Submission ● Module

4. Class: TestCase

Responsibilities	Collaborators
● Stores input parameters and expected	● Exercise

Responsibilities	Collaborators
output values. ● Maintains visibility status (public for pedagogical support vs. hidden for validation).	● VirtualJudge

5. Class: Submission

Responsibilities	Collaborators
● Stores the student's source code, chosen language, and timestamp. ● Records the final verdict (Accepted, Time Limit Exceeded, etc.). ● Logs execution metrics like time elapsed and memory consumed.	● Student ● Exercise ● VirtualJudge

6. Class: VirtualJudge

Responsibilities	Collaborators
● Receives the student's submission. ● Executes the code securely against all test cases. ● Enforces the exercise's time and memory limits strictly. ● Returns the final evaluation verdict.	● Submission ● TestCase ● Exercise

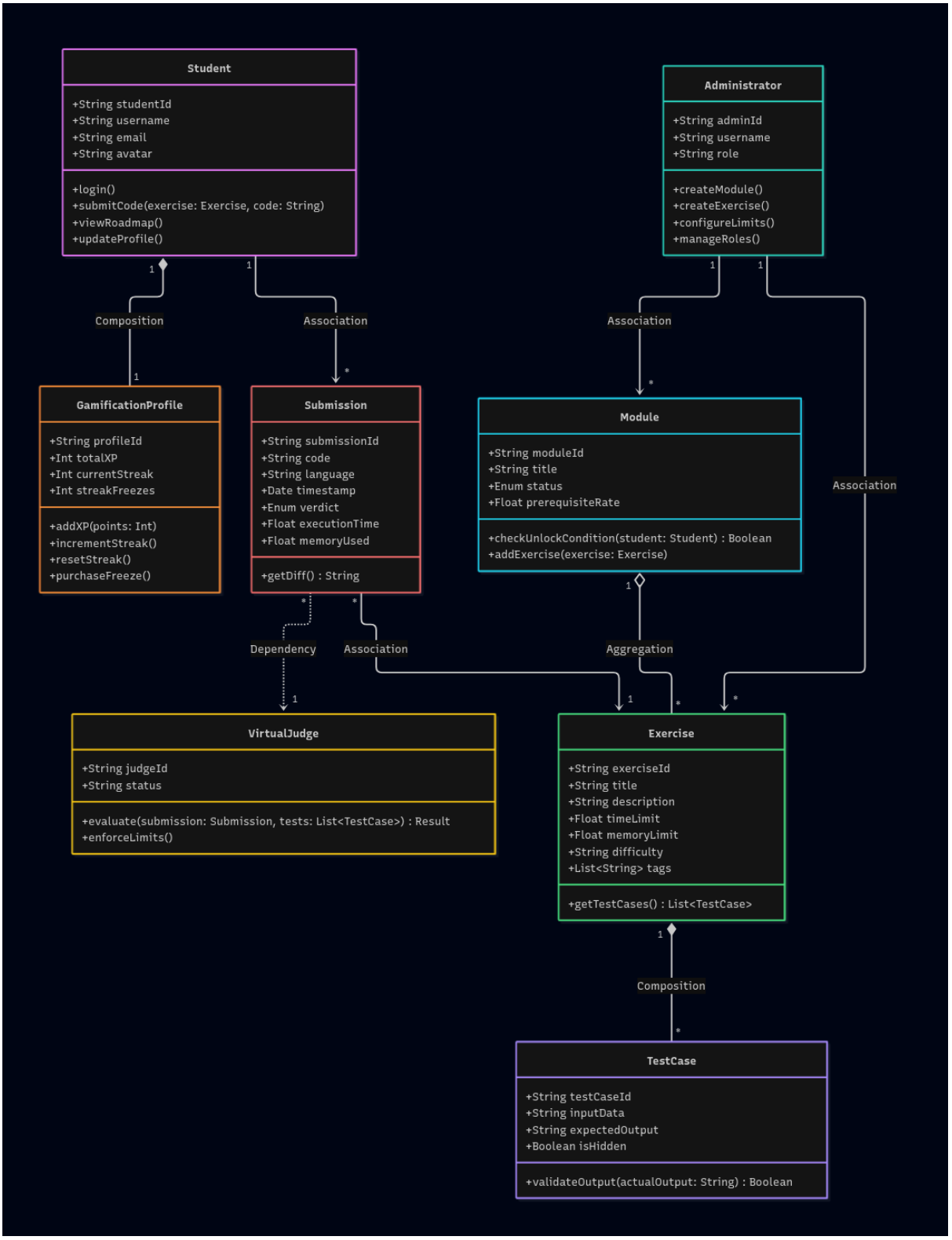
7. Class: GamificationProfile

Responsibilities	Collaborators
● Calculates and stores Experience Points (XP). ● Maintains daily activity streaks and streak freezes. ● Interfaces with the weekly leaderboard.	● Student

8. Class: Module

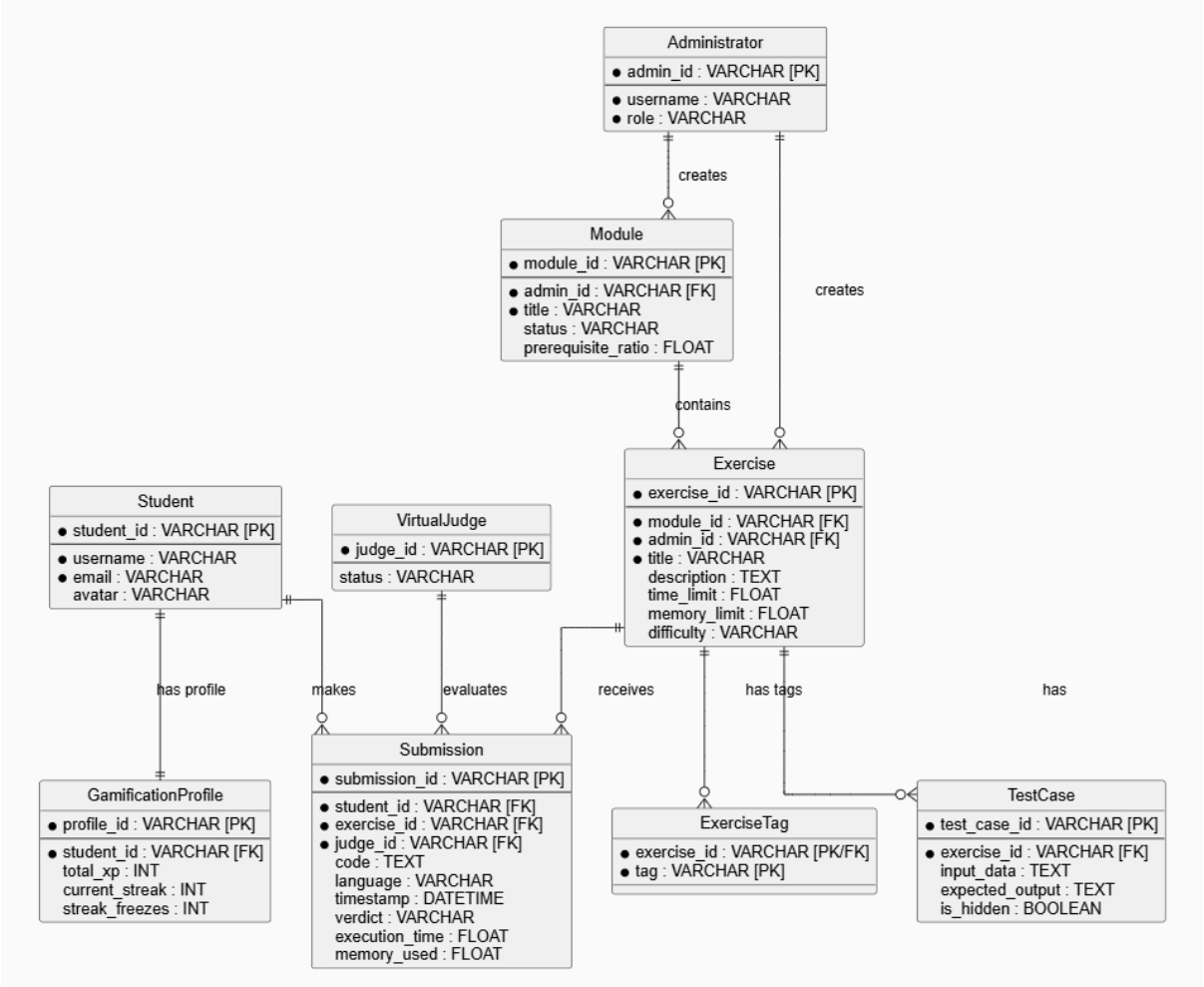
Responsibilities	Collaborators
● Groups theoretical lessons and practical exercises. ● Enforces prerequisite success rates (e.g., 80% effectiveness) before unlocking. ● Maintains status for the student (Locked, In Progress, Completed).	● Student ● Exercise

Part 3: Class Diagram (UML Representation)


Student
+String studentId
+String username
+String email
+String avatar
+login()
+submitCode(exercise: Exercise, code: String)
+viewRoadmap()
+updateProfile()
Administrator
+String adminId
+String username
+String role
+createModule()
+createExercise()
+configureLimits()
+manageRoles()
Composition
Association
Association
Association
GamificationProfile
+String profileId
+Int totalXP
+Int currentStreak
+Int streakFreezes
+addXP(points: Int)
+incrementStreak()
+resetStreak()
+purchaseFreeze()
Submission
+String submissionId
+String code
+String language
+Date timestamp
+Enum verdict
+Float executionTime
+Float memoryUsed
+getDiff() : String
Module
+String moduleId
+String title
+Enum status
+Float prerequisiteRate
+checkUnlockCondition(student: Student) : Boolean
+addExercise(exercise: Exercise)
Association
Dependency
Association
Aggregation
VirtualJudge
+String judgeId
+String status
+evaluate(submission: Submission, tests: List<TestCase>) : Result
+enforceLimits()
Exercise
+String exerciseId
+String title
+String description
+Float timeLimit
+Float memoryLimit
+String difficulty
+List<String> tags
+getTestCases() : List<TestCase>
Composition
TestCase
+String testCaseId
+String inputData
+String expectedOutput
+Boolean isHidden
+validateOutput(actualOutput: String) : Boolean

Part 4: Relational Database Model

The schema below targets PostgreSQL (Supabase) and is derived from the CRC cards, the class diagram and the Workshop 1 requirements. It has 13 tables in third normal form. The DDL was executed on PostgreSQL 16 to validate keys, types and constraints.



4.1 Table Definitions

Table 1: students

Column	Data type	Key	Constraints and description
student_id	VARCHAR	PK	Unique identifier for the student.
username	VARCHAR		Not null.
email	VARCHAR		NOT NULL, UNIQUE.
avatar	VARCHAR		Nullable. URL or path to avatar image.

Table 2: gamification_profiles — XP, streak and study-time counters of each student

Column	Data type	Key	Constraints and description
profile_id	VARCHAR	PK	Unique identifier for the profile.
student_id	VARCHAR	FK	→ `Student.student_id`. Enforces the 1-to-1 composition.
total_xp	INTEGER		DEFAULT 0.
current_streak	INTEGER		DEFAULT 0.
streak_freezes	INTEGER		DEFAULT 0.

Table 3: Administrator

Column	Data type	Key	Constraints and description
admin_id	VARCHAR	PK	Unique identifier for the admin.
username	VARCHAR		NOT NULL.
role	VARCHAR		NOT NULL.

Table 4: Module

Column	Data type	Key	Constraints and description
module_id	VARCHAR	PK	Unique identifier for the module. .
admin_id	VARCHAR	FK	→ `Administrator.admin_id`. Admin who created/manages it.
title	VARCHAR		NOT NULL.
status	VARCHAR		.
prerequisite_ratio	FLOAT		Percentage required to unlock.

Table 5: Exercise

Column	Data type	Key	Constraints and description
exercise_id	VARCHAR	PK	Unique identifier for the exercise.
module_id	VARCHAR	FK	→ `Module.module_id`. Module it belongs to.
admin_id	VARCHAR	FK	→ `Administrator.admin_id`. Admin who created it.

title	VARCHAR		NOT NULL.
description	TEXT		Not null. Full problem statement.
time_limit	FLOAT		Maximum execution time allowed.
memory_limit	FLOAT		Maximum memory usage allowed.
difficulty	VARCHAR		

Table 6: ExerciseTag

Column	Data type	Key	Constraints and description
exercise_id	VARCHAR	PK, FK	→ `Exercise.exercise_id`. Part of composite PK. .
tag_name	VARCHAR	PK	Part of composite PK. Describes the exercise topic.

Table 7: TestCase

Column	Data type	Key	Constraints and description
test_case_id	VARCHAR	PK	Unique identifier for the test case.
exercise_id	VARCHAR	FK	→ `Exercise.exercise_id`. Enforces composition.
input_data	TEXT		Data fed into the student's code.
expected_output	TEXT		Correct output for validation.
is_hidden	BOOLEAN		True if the case is kept secret from the student.

Table constraint: UNIQUE (exercise_id, hint_order).

Table 8: VirtualJudge

Column	Data type	Key	Constraints and description
judge_id	VARCHAR	PK	Unique identifier for the judging instance.
status	VARCHAR		NOT NULL, UNIQUE. Thematic label (e.g., loops, arrays).

Table 9: Submission

Column	Data type	Key	Constraints and description
submission_id	VARCHAR	PK	Unique identifier for the submission attempt.
student_id	VARCHAR	FK	→ `Student.student_id`. Student who submitted.

exercise_id	VARCHAR	FK	→ `Exercise.exercise_id`. Exercise being solved.
judge_id	VARCHAR	FK	→ `VirtualJudge.judge_id`. Judge handling evaluation.
code	TEXT		The source code submitted.
language	VARCHAR		Programming language used (e.g., 'Python', 'Java').
timestamp	DATETIME		Exact time of submission.
verdict	VARCHAR		Result (e.g., 'Accepted', 'Wrong Answer').
execution_time	FLOAT		Actual time taken to run.
memory_used	FLOAT		Actual memory consumed.

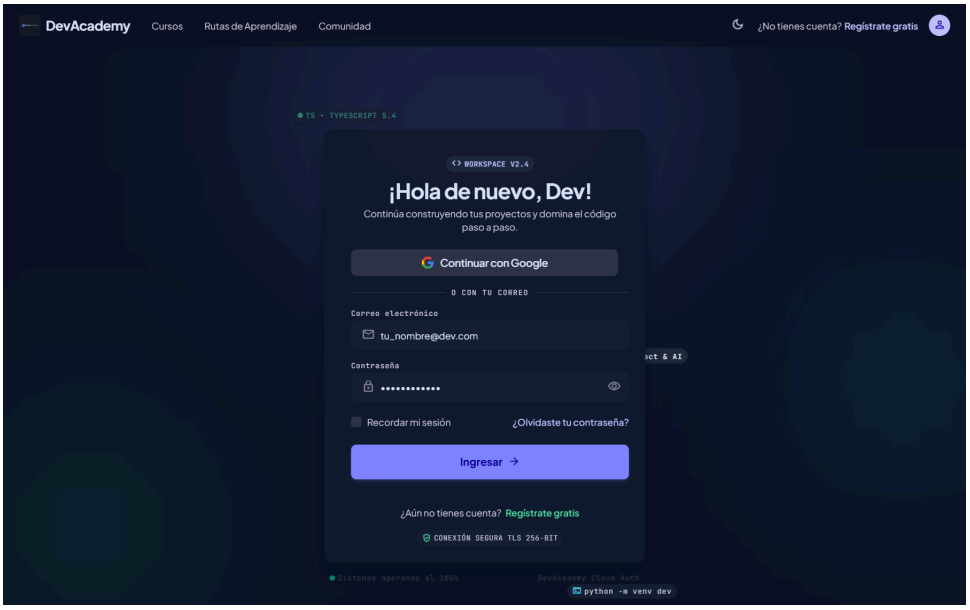
4.2 Relationships and Cardinalities

Cardinality is read parent : child. A left side of 1 means the foreign key is NOT NULL; 0..1 means it is nullable. The users–modules many-to-many relationship is resolved by module_progress, and exercises–tags by exercise_tags.

#	Parent table	Child table	Cardinality	FK column (in child)	ON DELETE
1	Student	GamificationProfile	1 : 0..1	student_id	CASCADE
2	Administrator	Module	0..1 : 0..*	admin_id	SET NULL
3	Administrator	Exercise	0..1 : 0..*	admin_id	SET NULL
4	Module	Exercise	1 : 0..*	module_id	CASCADE
5	Exercise	TestCase	1 : 0..*	exercise_id	CASCADE
6	Exercise	ExerciseTag	1 : 0..*	exercise_id	CASCADE
7	Student	Submission	1 : 0..*	student_id	CASCADE
8	Exercise	Submission	1 : 0..*	exercise_id	RESTRICT
9	VirtualJudge	Submission	0..1 : 0..*	judge_id	SET NULL

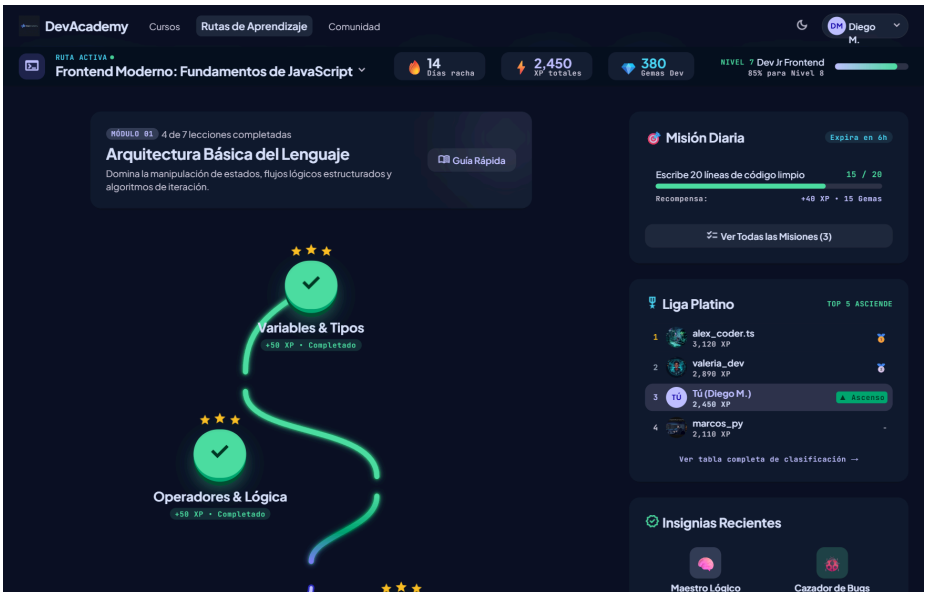
Part 5: Mockups

Mockup 1: Login



The image shows the login interface for the DevAcademy platform, which features a modern, dark-mode design with a top navigation bar that includes the logo, as well as links to courses and learning paths. In the center is an authentication card that offers quick access via Google (OAuth 2.0), along with standard fields for entering an email address and password, and a primary button for logging in securely.

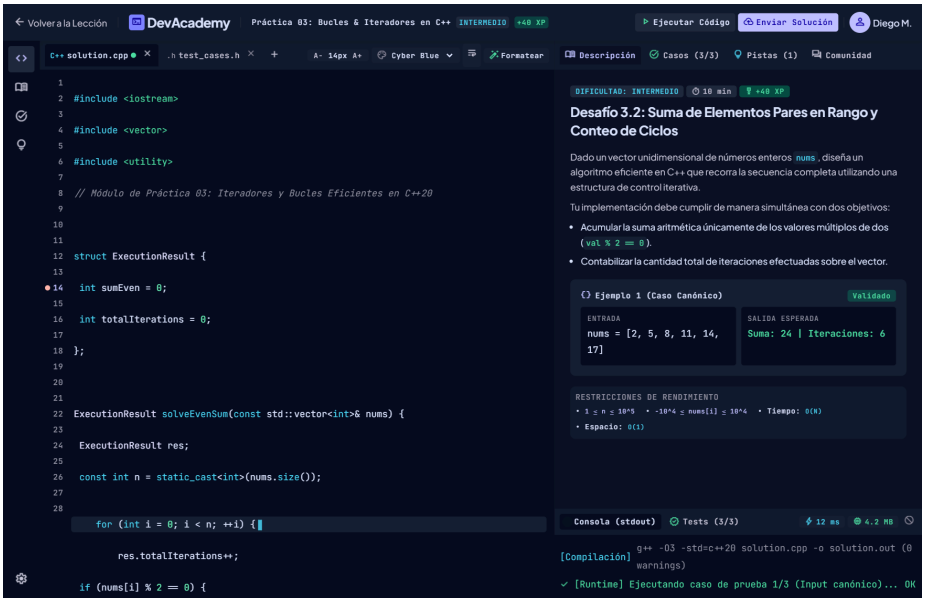
Mockup 2: Learning path



The image shows the main screen of the interactive learning path on the DevAcademy platform. It features a gamified, dark-mode interface that includes a top bar displaying user statistics (streak days, XP points, and current level), a central tree-like map

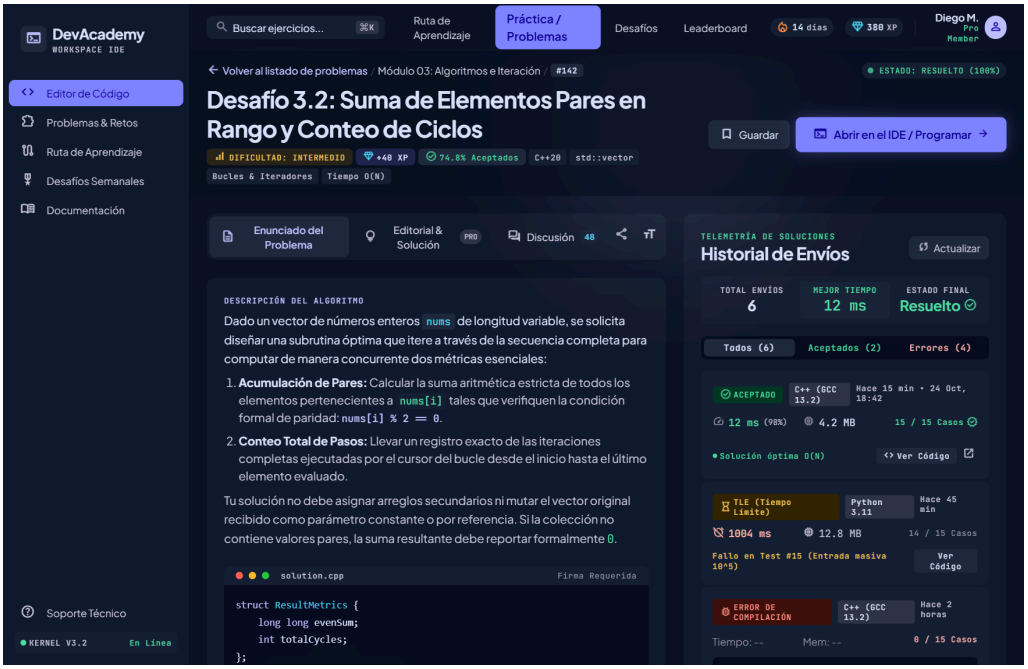
with nodes and completed lessons—such as "Variables" and "Operators"—as well as side panels containing daily quests, a weekly leaderboard, and recent badges.

Mockup 3: Development code



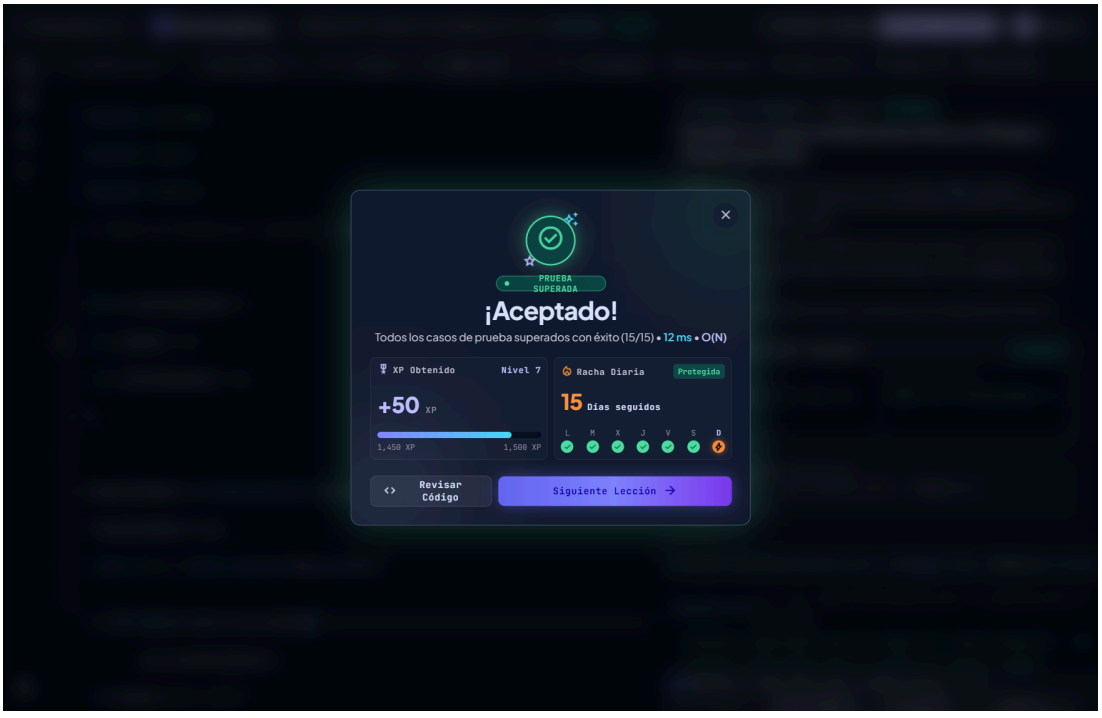
The image shows the DevAcademy practice and development environment, which features an IDE divided into three main sections: a code editor on the left with support for C++, customizable typography, and line numbering; an upper-right panel containing problem instructions, time/memory constraints, and sample test cases; and a console at the bottom that displays execution results, response times in milliseconds, and memory usage.

Mockup 4: Shipping history



The image shows the individual view of a problem on DevAcademy, featuring a side navigation menu, the detailed problem statement—including requirements and the code signature—on the left, and a telemetry panel on the right displaying the student's submission history; this panel shows performance metrics (time in ms and memory) alongside color-coded verdicts such as "Accepted," "TLE (Time Limit Exceeded)," and "Compilation Error."

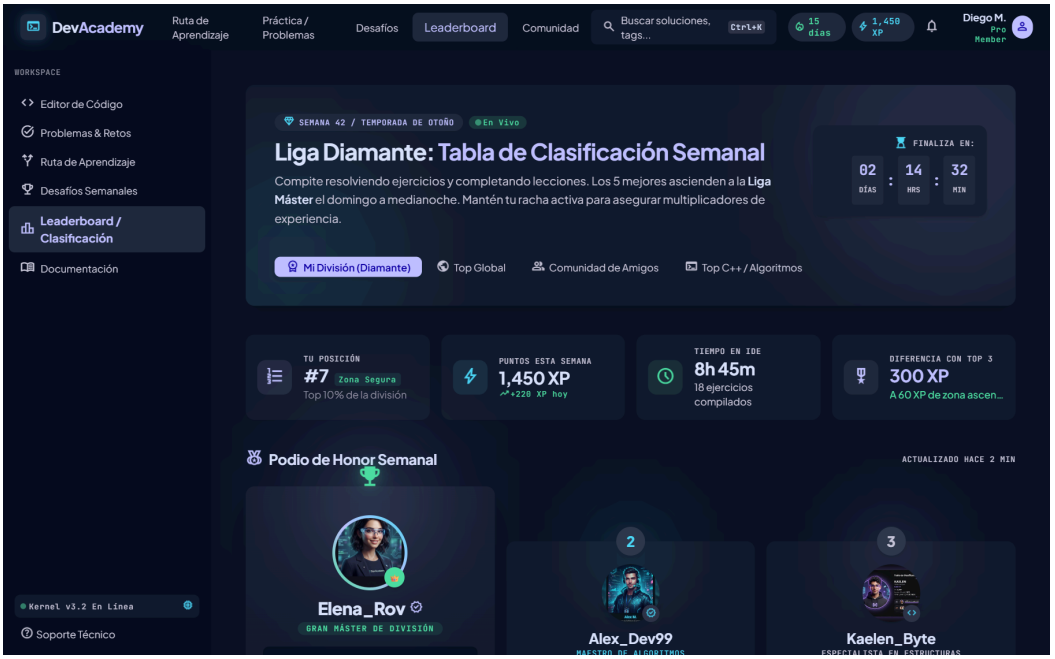
Mockup 5: Code accepted



The image shows a DevAcademy gamification pop-up that congratulates the user upon the successful completion of an exercise ("Accepted!"), noting that they passed all test

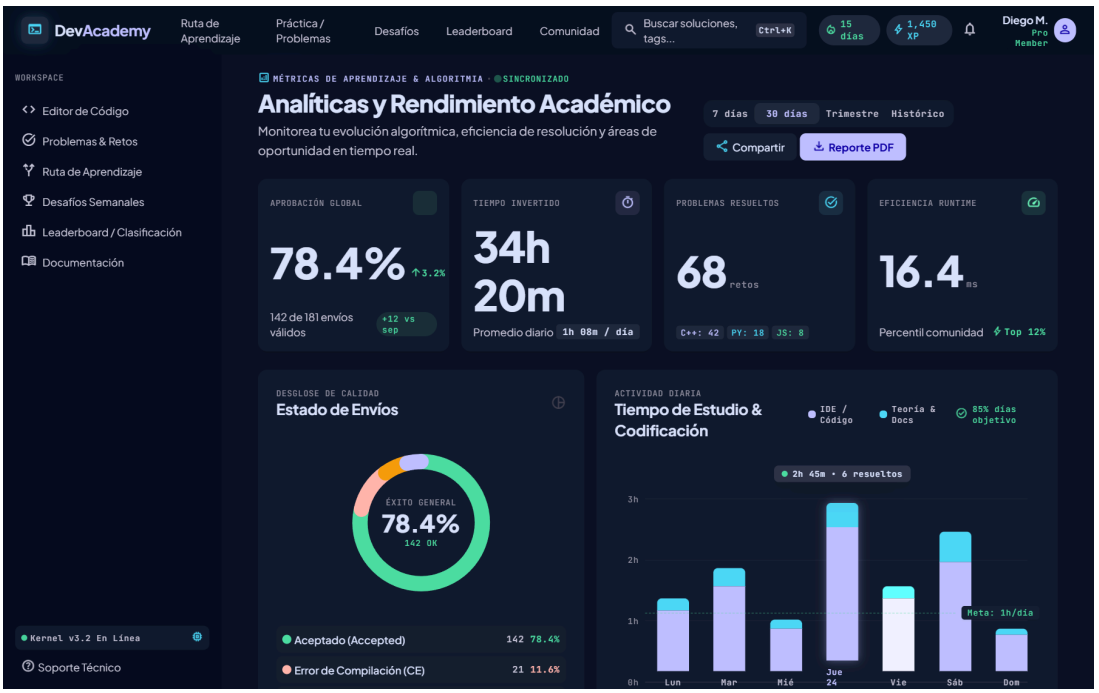
cases (15/15) in 12 ms, awarding them +50 Experience Points (XP) toward their level progression, and updating their daily streak to 15 consecutive days.

Mockup 6: Qualifying ligue



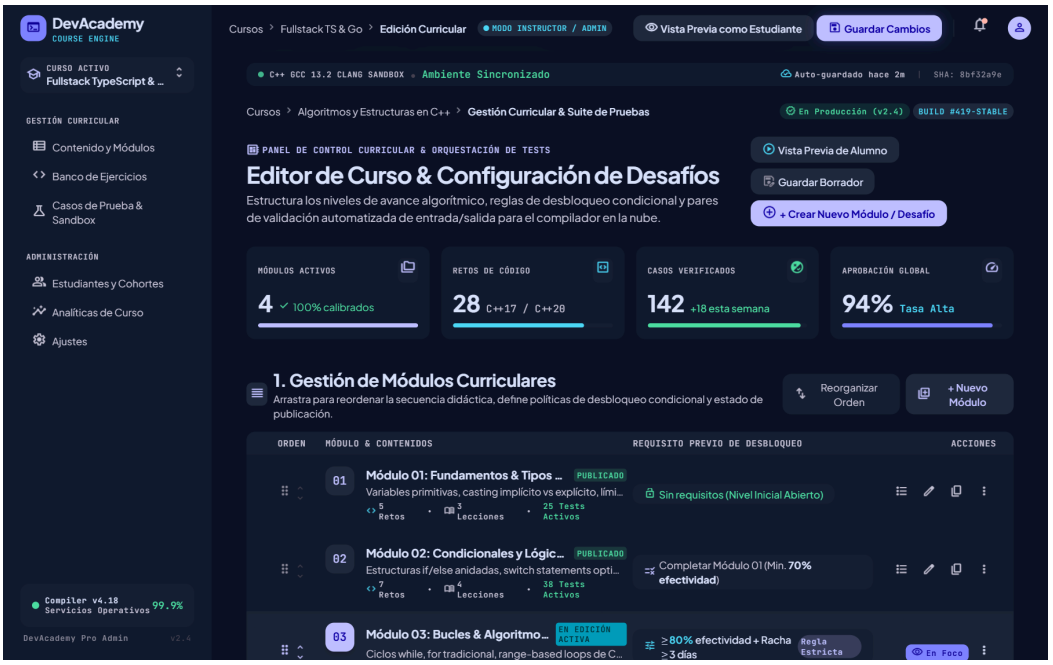
The image shows the DevAcademy weekly leaderboard screen (Diamond League), which includes a countdown timer until the weekly reset, personal student metrics—such as current rank (#7 in the Safe Zone), XP accumulated during the week (1,450 XP), active work time in the editor (8h 45m), and the gap to reach the top 3—alongside the weekly podium highlighting the division's top competitors.

Mockup 7: Student statistics



The image shows DevAcademy's academic performance and analytics dashboard, which summarizes student progress through key indicators such as the overall pass rate (78.4%), total study time (34h 20m), number of problems solved (68 challenges), and execution efficiency. It also includes a pie chart displaying the success rate and the distribution of submission outcomes, alongside a bar chart detailing daily active coding time.

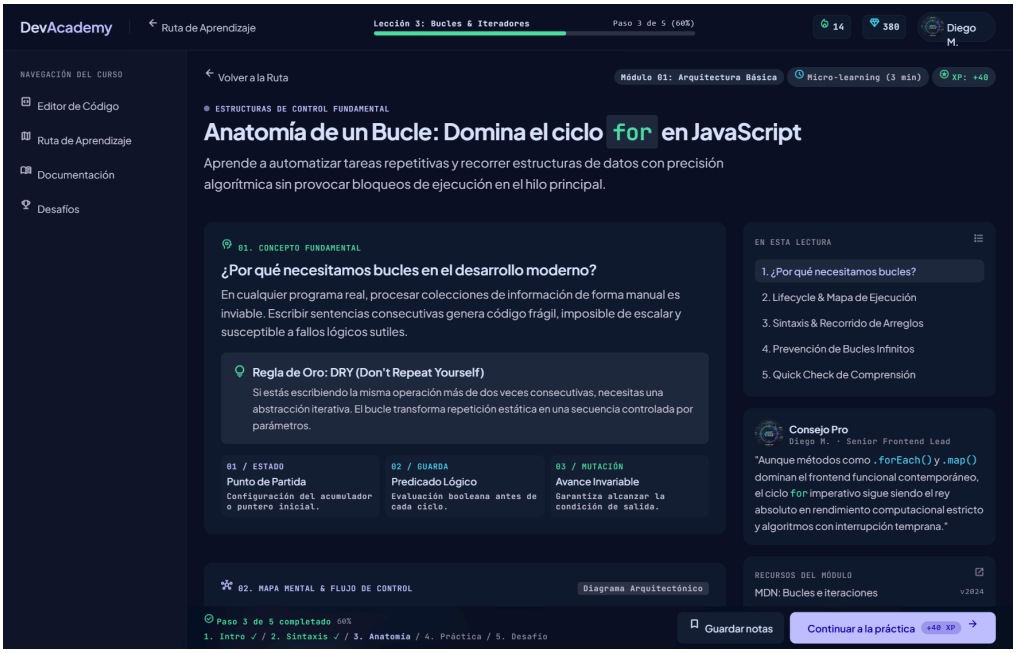
Mockup 8: Dashboard admin



The image shows the DevAcademy administration panel, focused on curriculum management and challenge configuration. It allows instructors to create and organize modules and lessons, define global course metrics (such as coding challenges and verified

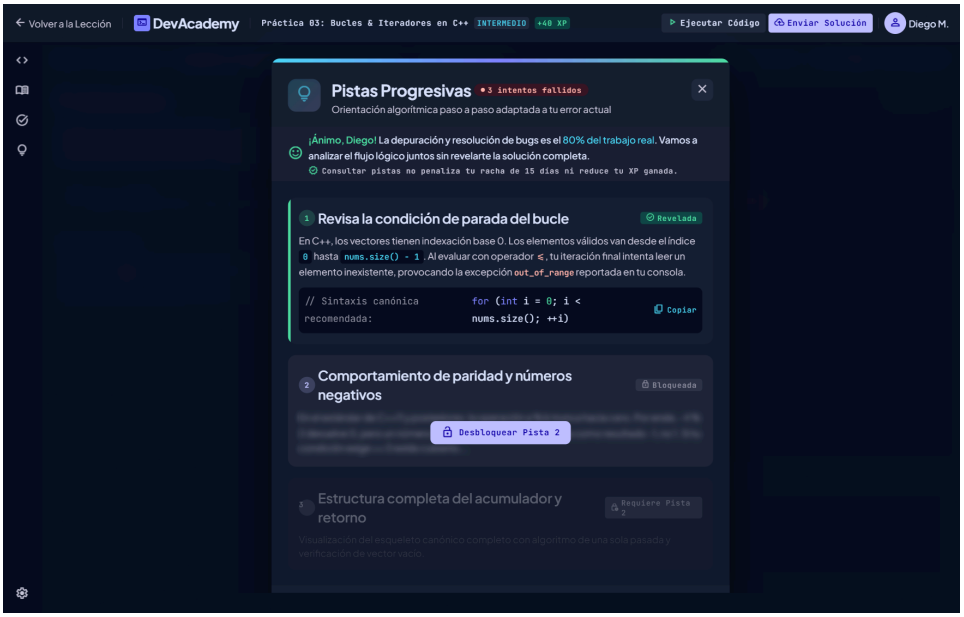
test cases), and set up conditional prerequisites for unlocking topics (for example, requiring an 80% success rate to progress).

Mockup 9: Educational lesson



The image shows a theoretical lesson from DevAcademy, designed in a micro-learning format. It presents content in short, focused blocks covering fundamental concepts—such as the `for` loop in JavaScript—and features an interactive table of contents for the lesson on the right, alongside a box containing expert tips. A top bar displays the student's current streak, experience points (XP), and a progress indicator, while a floating button at the bottom allows the user to "Continue to practice."

Mockup 10: Clues for the student



The image shows the "Progressive Hints" pop-up panel in DevAcademy, which is triggered after the system detects multiple failed attempts (3 attempts) by a student. It provides step-by-step algorithmic guidance tailored to the specific error, without penalizing the student's XP or their daily streak. The interface displays an initial hint containing detailed explanations regarding C++ syntax and the loop's termination condition, while subsequent hints remain locked—offering the option to unlock them progressively to guide the learning process without revealing the complete solution all at once.

6. Business Model Processes

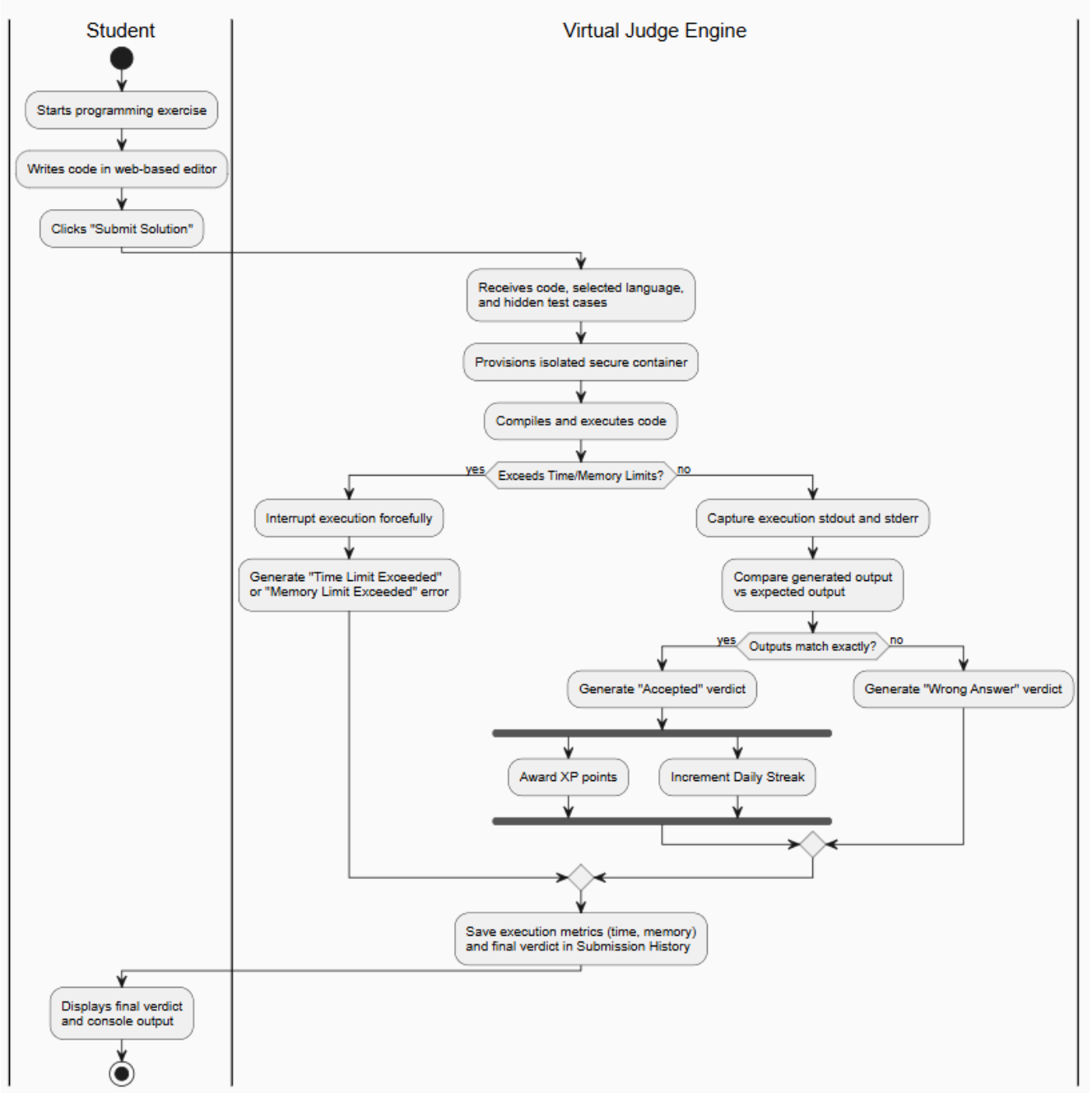
Core Business Process: Code Submission and Automated Evaluation

Role in the Application: This process represents the core value proposition of the platform: evaluating the student's programming logic automatically and securely. It is vital because it provides objective, immediate feedback to the student without requiring any human intervention.

Process Description:

1. The flow begins when the student finishes writing their solution in the web editor and clicks "Submit Solution".
2. The system packages the code and routes it to the Virtual Judge Engine.
3. The engine runs the code against public and strictly hidden test cases inside an isolated container, measuring execution time and memory consumption in real-time.
4. **Constraint Check:** If the code breaks the technical constraints, the engine forcefully halts execution and returns a "Time/Memory Limit Exceeded" verdict.
5. **Validation:** If the code executes successfully within the limits, the system compares the outputs.
6. **Gamification:** A perfect match yields an "Accepted" verdict, which triggers parallel gamification updates (awarding XP points and updating the daily streak). If they do not match, it yields a "Wrong Answer" verdict.

7. Finally, the submission metrics are saved in the history, and the final verdict is displayed on the student's screen

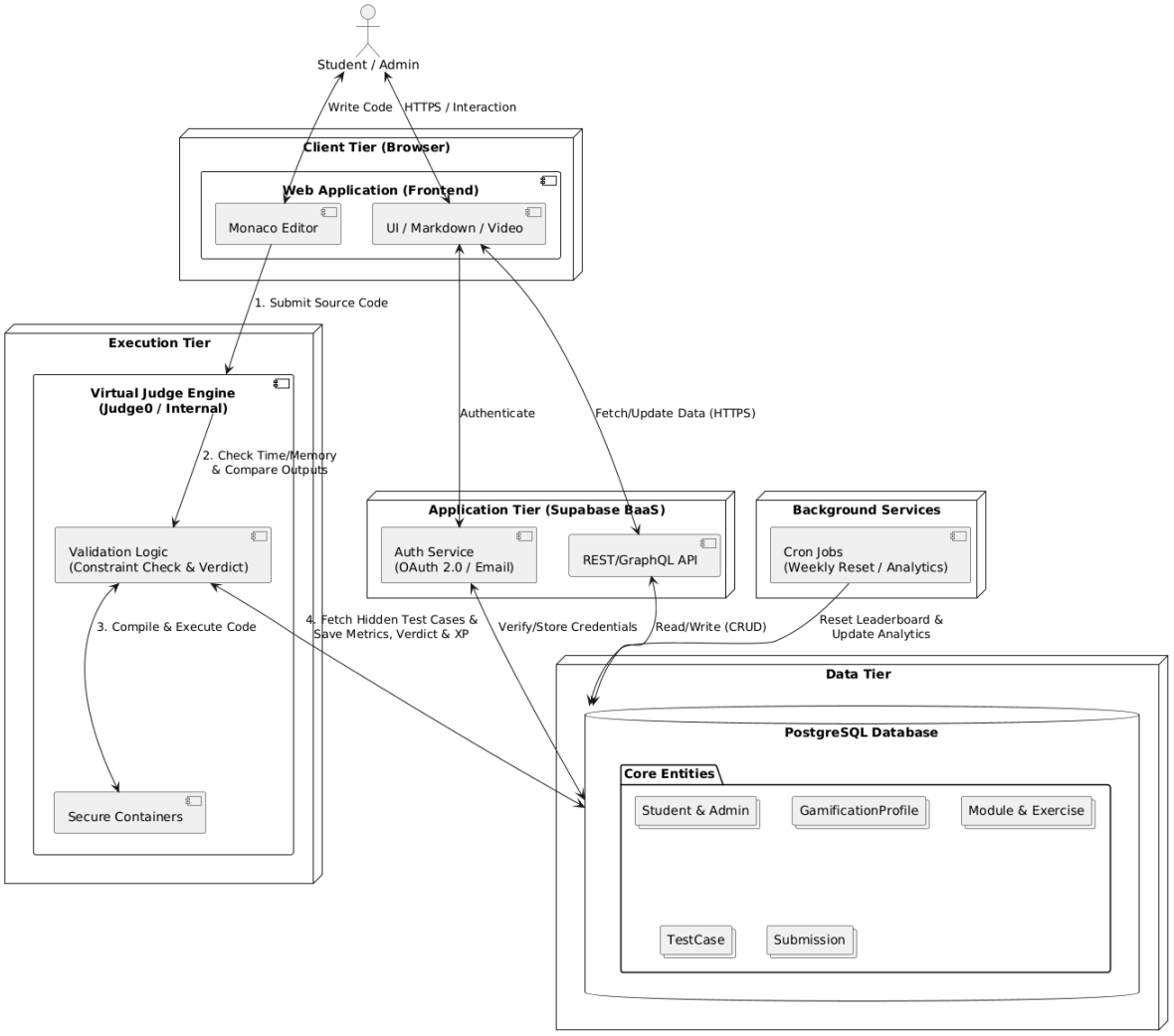


7. Architecture Diagram

The software architecture of DevAcademy follows a modular, cloud-based approach to ensure scalability, security, and real-time responsiveness. The system is composed of the following main layers and components:

- **Client / Presentation Tier:** A responsive web application built for browsers. It features a UI that renders Markdown and embedded multimedia (e.g., YouTube videos). It integrates the Monaco Editor to provide an interactive, in-browser coding environment with syntax highlighting and auto-completion.
- **Application & Authentication Tier:** Powered by Supabase (Backend-as-a-Service), this layer handles secure user authentication using both traditional encrypted passwords and the OAuth 2.0 protocol for Google sign-ins. It exposes a REST/GraphQL API for seamless communication between the client and the database.
- **Data Tier:** A PostgreSQL relational database, managed via Supabase, securely stores all platform data. It relies on normalized core entities including Student, Administrator, GamificationProfile, Module, Exercise, TestCase, and Submission.
- **Execution Tier (Virtual Judge Engine):** The core component responsible for automated code evaluation. In the initial releases, it relies on the Judge0 API, while Release 3 will integrate a self-hosted Internal Virtual Judge to guarantee operational autonomy. It provisions secure, isolated containers to run student submissions against hidden test cases. It implements strict validation logic to perform a constraint check (enforcing time and memory limits) before comparing the generated output.
- **Background Services:** Scheduled tasks (Cron Jobs) run asynchronously to perform maintenance. These jobs are responsible for resetting the weekly leaderboard counters and consolidating academic analytics without requiring manual intervention.

Communication Flows: The student interacts with the web interface via HTTPS. The UI communicates with the Auth service to verify credentials and accesses the database via API to retrieve the learning roadmap and gamification profile. When a student submits a solution, the source code is sent to the Virtual Judge Engine. The engine fetches the hidden test cases from the database and provisions an isolated container. During execution, the engine performs a Constraint Check; if limits are exceeded, it forces an interruption (Time/Memory Limit Exceeded). Otherwise, it validates the generated output against the expected output. Finally, the database is updated with the Submission record and, if the verdict is "Accepted", the system awards XP points and increments the daily streak in the GamificationProfile. Background Cron Jobs independently update weekly statistics.



AI Usage Declaration

Gemini was used to brainstorm ideas, refine English phrasing, and generate the PlantUML diagram code. All final content was reviewed and validated by the author.

jnmrtnz111. (n.d.). *Workshop-2* [Source code directory]. In the *learn_2_code* repository. GitHub. Retrieved October 16, 2026, from https://github.com/jnmrtnz111/learn_2_code/tree/main/Workshop-2